# Vim or Emacs: A Complete Guide to Choosing

The friendly editor war explained, with a practical plan to decide.

## Contents


1 Introduction: a friendly editor war
2 Origins and history
3 Core philosophy: modal versus modifier
4 The editing model in practice
5 Configuration and extension languages
6 Plugins and signature features
7 Performance, startup and footprint
8 Ubiquity and remote work
9 Modern variants and the hybrid path
10 The learning curve and resources
11 Typical users and workflows
12 Common myths, examined
13 How to decide: a checklist
14 A two-week trial plan
15 Frequently asked questions
16 Glossary of terms

# Part 1 — Introduction: a friendly editor war

Vim and Emacs are two of the most powerful and beloved text editors ever made. For decades, programmers have playfully argued about which is better, in what is fondly called the “editor war.” The friendly truth is that both are superb, both reward the effort you put in, and the right choice depends on how you like to work.

This guide explains where each comes from, how their philosophies differ, what daily use feels like, how they extend and customize, and how to choose. It ends with a decision checklist and a two-week trial plan so you can decide from experience rather than hearsay.

> Good news up front: thanks to modern variants you can even combine the best of both, so you are not locked into a lifelong allegiance.

# Part 2 — Origins and history

Vim descends from an older editor called vi, which has been part of Unix systems for a very long time. Vim, meaning “Vi IMproved,” added many features while keeping vi’s efficient, keyboard-driven spirit. Because vi is everywhere on Unix-like systems, knowing Vim is a practical, transferable skill.

Emacs is equally venerable. It was built around a small core and a powerful programming language, so that almost everything about it can be inspected and changed. Over the years people have used Emacs not just to edit text but to read email, manage projects, browse files and much more.

# Part 3 — Core philosophy: modal versus modifier

The deepest difference is how you interact with the editor.

## Vim and modal editing

Vim is **modal**. You switch between modes, such as normal mode for navigating and manipulating text, insert mode for typing, and visual mode for selecting. In normal mode, single letters become powerful commands, so you can edit with very few keystrokes once the motions become second nature.

## Emacs and modifier keys

Emacs is **non-modal**. You are always typing text, and you trigger commands by holding modifier keys, often in short chords. This is immediately intuitive in one sense, since letters always insert letters, but the chords take practice and have inspired the affectionate phrase “Emacs pinky.”

Neither model is objectively superior. Modal editing feels astonishingly efficient to its fans; the always-insert model feels natural and direct to its fans.

# Part 4 — The editing model in practice

In Vim, you might press a few keys to delete a word, change the text inside quotes, or jump to a matching bracket, all without leaving the home row. The commands combine like a little language of verbs and motions, which is why experienced users edit so quickly.

In Emacs, the same operations exist, reached through key chords and named commands. Emacs also makes it easy to discover and run any command by name, and to see what each key does, which many learners find welcoming.

- Vim rewards memorizing a compact vocabulary of motions and operators.
- Emacs rewards learning its discoverable command system and gradually customizing it.

# Part 5 — Configuration and extension languages

Both editors are deeply configurable, but through different languages.

- **Vim** is configured with Vimscript, and modern **Neovim** also supports the Lua language, which is fast and pleasant for writing plugins.
- **Emacs** is configured and extended with Emacs Lisp, a complete programming language. This is why Emacs can be reshaped so thoroughly: large parts of it are written in the same language you use to customize it.

If you enjoy programming your tools, Emacs offers a uniquely open canvas. If you want powerful customization with a gentler on-ramp, Neovim's Lua ecosystem is very approachable.

# Part 6 — Plugins and signature features

### *Emacs highlights*

- **Org-mode** is a celebrated system for notes, planning, outlining and even publishing, and many people adopt Emacs just for it.
- **Magit** is widely praised as one of the best interfaces to the Git version-control system anywhere.
- Emacs can host email, terminals, file management and more, becoming a single integrated workspace.

### *Vim highlights*

- A vast plugin ecosystem covers file navigation, fuzzy finding, status lines and more.
- Modern Neovim has first-class support for language servers, bringing smart completion and diagnostics.
- The editing core is so good that many other editors offer a “Vim mode” to imitate it.

## Part 7 — Performance, startup and footprint

- Vim is lightweight and starts almost instantly, which is perfect for quick edits and for working over remote connections.
- Emacs is heavier and historically slower to start, but it solves this elegantly with a background server, or daemon, so new windows open instantly.
- For editing a single file in a hurry, Vim feels effortless; for living in one rich environment all day, Emacs’s weight is easy to justify.

# Part 8 — Ubiquity and remote work

Vi, and therefore Vim-style editing, is available on virtually every Unix and Linux server. If you administer machines or frequently connect to remote systems, being comfortable in Vim is a genuinely useful, portable skill. Emacs is wonderful but less universally pre-installed, so on an unfamiliar server you may reach for vi by necessity.

# Part 9 — Modern variants and the hybrid path

The old rivalry has softened, because modern projects let you mix the strengths of each.

- **Neovim** is a modern take on Vim with Lua support, a healthy plugin scene and strong language-server integration.
- **Doom Emacs** and **Spacemacs** are popular Emacs configurations that come ready-made and polished.
- **evil-mode** brings Vim's modal editing into Emacs, so you can have Vim keys and Emacs power at once.

This means the decision is rarely final. Many people enjoy Vim keybindings inside an Emacs environment, or a modern Neovim setup that feels like a full development environment.

# Part 10 — The learning curve and resources

Both editors ask for an investment, and both repay it. The first days can feel awkward, but the awkwardness fades and is replaced by speed and comfort.

- Vim ships with an interactive tutorial; running it is the classic first step.
- Emacs has a built-in tutorial and an extensive, discoverable help system.
- Go slowly: learn a handful of commands, use them until they are automatic, then add a few more.
- Resist installing dozens of plugins on day one; grow your setup as real needs appear.

# Part 11 — Typical users and workflows

### *People who tend to love Vim*

- System administrators and anyone who works on remote servers often.
- Programmers who value speed, minimalism and keeping their hands on the keyboard.
- People who like a tool that starts instantly and stays out of the way.

### *People who tend to love Emacs*

- Those who want one extensible environment for code, notes, planning and version control.
- People who enjoy customizing and even programming their tools.
- Writers and researchers drawn to Org-mode for organizing their work.

# Part 12 — Common myths, examined

***“You must pick one forever.”***

Not so. With evil-mode and modern setups you can blend both, and you can switch later without losing your skills.

***“They are obsolete now.”***

Both remain actively developed and widely used, and their editing ideas influence many newer editors.

***“Emacs is bloated and Vim is primitive.”***

Emacs is large because it is capable, and you use only what you need. Vim is lean by choice and is far more powerful than its minimalist appearance suggests.

## Part 13 — How to decide: a checklist

1. Do you work on remote servers often? That favors Vim-style skills.
2. Do you want one integrated environment for many tasks? That favors Emacs.
3. Do you enjoy programming your tools in a Lisp language? Emacs will delight you.
4. Do you want instant startup and minimalism? Vim, or Neovim, fits.
5. Do you want both? Try Neovim, or Emacs with evil-mode.

# Part 14 — A two-week trial plan

The fairest way to choose is to live with each for a short while.

- **Days 1 to 3:** work through Vim’s built-in tutorial and use Vim for small, real edits.
- **Days 4 to 6:** work through the Emacs tutorial and use Emacs for similar tasks.
- **Days 7 to 10:** pick the one that felt more natural and use it for everything, adding one or two helpful plugins.
- **Days 11 to 14:** try a hybrid, such as evil-mode in Emacs or a modern Neovim setup, and decide.

# Part 15 — Frequently asked questions

### *Which is faster to edit in?*

Skilled users of either are very fast. Vim’s modal model gives a famous sense of speed, but a well-configured Emacs is no slouch.

### *Which should a beginner learn first?*

Learning basic Vim is valuable because vi is everywhere. From there you can explore Emacs if its environment appeals to you.

### *Can I use these for non-programming writing?*

Absolutely. Many writers love Vim for distraction-free editing and Emacs for Org-mode’s organizing power.

# Part 16 — Glossary of terms

- **Modal editing** — switching between modes, as in Vim, so keys mean different things in each mode.
- **Modifier key** — a key such as Control or Alt held to trigger commands, central to Emacs.
- **Vimscript / Lua** — languages used to configure Vim and Neovim.
- **Emacs Lisp** — the language used to configure and extend Emacs.
- **Org-mode** — an Emacs system for notes, outlining and planning.
- **Magit** — an acclaimed Emacs interface to the Git version-control system.
- **evil-mode** — an Emacs package that provides Vim-style modal editing.
- **Daemon** — a background process; Emacs uses one so new windows open instantly.